

Tic Tac Toe Online

עמרם נועה- 323014019
סימה פרילוק - 327441101

1. מבוא

פרויקט Tic Tac Toe Online הוא משחק איקס-עיגול מרובה משתתפים הפועל ברשת, וממומש בשפת Python תוך שימוש ב־ PyQt5 לממשק גרפי וב־ TCP Sockets לתקשורת בין לקוח לשרת. הפרויקט פותח במסגרת קורס "רשתות מחשבים ואינטרנט Computer Networks and Internet"

2. סקירת המערכת

המערכת מחולקת לשלושה רכיבים עיקריים:

- **Common** - קוד משותף ללקוח ולשרת (פרוטוקול, לוגיקת משחק, מודלים)
- **Server** - שרת מרכזי המנהל חיבורים, משחקים ומצב
- **Client** - יישום גרפי דרכו המשתמש משחק ומתקשר עם השרת

תכונות עיקריות:

- תמיכה ב-2 עד 10 שחקנים: המשחק מתאים את עצמו למספר המשתתפים
- לוח דינמי: גודל הלוח נקבע מספר השחקנים 2^{1+} .
- ניהול תהליכונים: טיפול במספר משחקים ולקוחות במקביל.
- ממשק משתמש מקצועי: מבוסס Qt5 לחוויית משתמש חלקה

3. ארכיטקטורה כללית

המערכת בנויה בארכיטקטורת **Client-Server**:

- השרת מאזין לחיבורים נכנסים ומנהל מספר משחקים במקביל
- כל לקוח מתחבר לשרת, מצטרף למשחק ושולח מהלכים
- השרת מאמת מהלכים, מעדכן מצב ומשדר אותו לכל השחקנים

3.1 : טכנולוגיות

Python שפת הליבה לפיתוח הלוגיקה והשרת
PyQt5 בניית ממשק המשתמש ותצוגת הלוח
Sockets (TCP) תקשורת אמינה.
Threading - ניהול פעולות מקבילות למניעת חסימת GUI
JSON - סיראליזציה של הודעות למעבר בתוך ה-Socket.

4. תיעוד מודולים משותפים (common)

- **protocol.py** - הגדרת סוגי ההודעות ושיטות קידוד/פענוח.
- **player.py** - ייצוג שחקן (מזהה, שם, סמל וסטטוס פעיל).
- **board.py** - לוגיקת הלוח (בדיקת חוקיות מהלך, זיהוי ניצחון ותיקו).
- **game.py** - ניהול סבב המשחק (ניהול תורות, מעבר בין מצבי משחק)

5. מודולי שרת (server)

5.1 **server.py** - שרת ראשי : המחלקה המרכזית של השרת.

תפקידים:

- פתיחת Socket והאזנה
- קבלת חיבורים
- ניתוב הודעות
- שידור עדכוני מצב
- כל לקוח מטופל ב־ Thread נפרד.

5.2 **client_handler.py** - טיפול בלקוח - אחראי על תקשורת עם לקוח בודד.

תפקידים:

- קבלת הודעות
- פענוח JSON
- טיפול בניתוק

5.3 **game_manager.py** - מנהל משחקים - אחראי על כל המשחקים הפעילים בשרת.

יכולות:

- יצירת משחקים
- מיפוי שחקנים למשחקים
- ניקוי משחקים שהסתיימו

6. מודולי לקוח (client)

6.1 **network_client.py** - תקשורת רשת - מחלקה המטפלת בתקשורת מול השרת.

מאפיינים:

- Thread ייעודי לקליטת הודעות
- שימוש ב־ Qt Signals לעדכון ה־GUI
- שליחת בקשות לשרת

6.2 **board_widget.py** - תצוגת לוח - Widget גרפי ללוח המשחק.

יכולות:

- יצירת Grid דינמי
- קליקים על תאים
- הדגשת מהלכים וניצחון

6.3 main_window.py - חלון ראשי- אחראי על ממשק המשתמש הראשי.

כולל:

- חיבור לשרת
- לובי משחקים
- מסך משחק
- הצגת הודעות וסטטוס

7. פירוט מודולים ולוגיקה

7.1 לוגיקה בצד השרת

השרת פועל במודל Multi-threaded המאפשר טיפול בלקוחות רבים ובמספר משחקים מקבילים

- **ניהול חיבורים: (server.py)** השרת מאזין בפורט המוגדר (ברירת מחדל 5555). עבור כל חיבור חדש, נוצר אובייקט ClientHandler המריץ תהליכון (Thread) ייעודי.
- **עיבוד פקודות:**
 - **CREATE_GAME** - הלקוח מגדיר את כמות השחקנים N. השרת מאתחל אובייקט Game עם לוח דינמי בגודל N^2+1
 - **MOVE** - השרת מקבל קואורדינטות (row, col) מוודא שהמהלך חוקי (המשבצת פנויה והתור נכון) ומבצע עדכון.
 - **Broadcasting** - לאחר כל מהלך תקין, השרת שולח הודעת GAME_STATE לכלל השחקנים בחדר כדי לסנכרן את התצוגה אצל כולם בזמן אמת.
- **זיהוי סיום משחק:** השרת מריץ אלגוריתם ב board.py-הסורק את הלוח לזיהוי רצף של בדיוק 3 סימנים (אופקית, אנכית או אלכסונית) או זיהוי תיקו כשהלוח מלא.

7.2 לוגיקה בצד הלקוח

- הלקוח מנהל הפרדה בין שכבת התצוגה לשכבת התקשורת כדי למנוע קיפאון של הGUI.
- **תקשורת אסינכרונית (network_client.py):** מחלקה זו מריצה תהליכון האזנה (Listener Thread) שקולט הודעות מהשרת ללא הפסקה. כאשר מתקבלת הודעה (כמו עדכון לוח), היא נשלחת ל GUI באמצעות PyQt Signals
 - **תצוגה דינמית (board_widget.py):** הרכיב הגרפי בונה רשת (Grid) של תאים לחיצים בהתאם לגודל הלוח שהתקבל מהשרת. הוא מנהל ויזואלית את השבתת התאים כשזה לא תורו של השחקן.
 - **ניהול מצבים (main_window.py):** מחליף בין תצוגת הלובי (רשימת משחקים) לתצוגת המשחק הפעיל באמצעות QStackedWidget

8. טיפול במקרי קצה

המערכת כוללת מנגנוני הגנה למניעת קריסות והבטחת חוויית משחק תקינה:

- **ולידציה קשיחה בשרת:** גם אם הלקוח נפרץ או "מרמה", השרת בודק כל מהלך מול המצב הלוגי השמור אצלו ודוחה מהלכים לא חוקיים (כגון מהלך מחוץ לתור)
- **ניהול ניתוקים:** השרת מזהה ניתוק (נפילת SOCKET) ומפעיל פונקציית Cleanup ב-game_manager שמעדכנת את שאר השחקנים ומסירה את המשחק במידת הצורך.

- בטיחות תהליכונים (Thread Safety) : שימוש ב-Locks-בשרת בעת גישה למבני נתונים משותפים (כמו רשימת המשחקים) למניעת Race Condition .
- תגובתיות הממשק: שימוש ב try-except-בעת פענוח JSON מבטיח שהלקוח לא יקרוס אם התקבלה הודעה פגומה מהרשת.

9. פרוטוקול תקשורת

כל ההודעות מועברות כ־JSON ומופרדות ב־\

שלב	פעולת לקוח	תגובת שרת	שידור לכולם
1.	התחברות לשרת	אישור חיבור	-
2.	יצירת משחק (CREATE_GAME)	אישור עם game_id	-
3.	הצטרפות(JOIN_GAME)	אישור הצטרפות	הודעה לכל השחקנים בחדר
4.	ביצוע מהלך (MAKE_MOVE)	אימות ועדכון	שליחת מצב לוח חדש לכולם
5.	סיום משחק	-	הודעת GAME_OVER לכולם

10. הפעלה והתקנה

דרישות מערכת Python 3.8+ ; Windows 10 & PyQt5

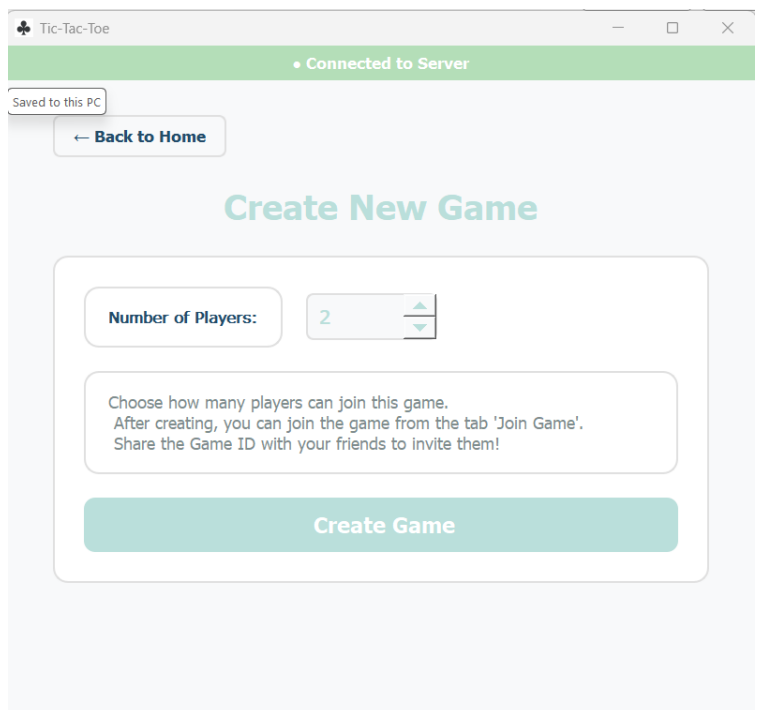
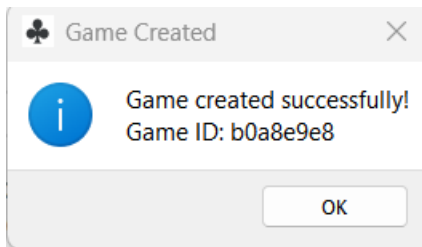
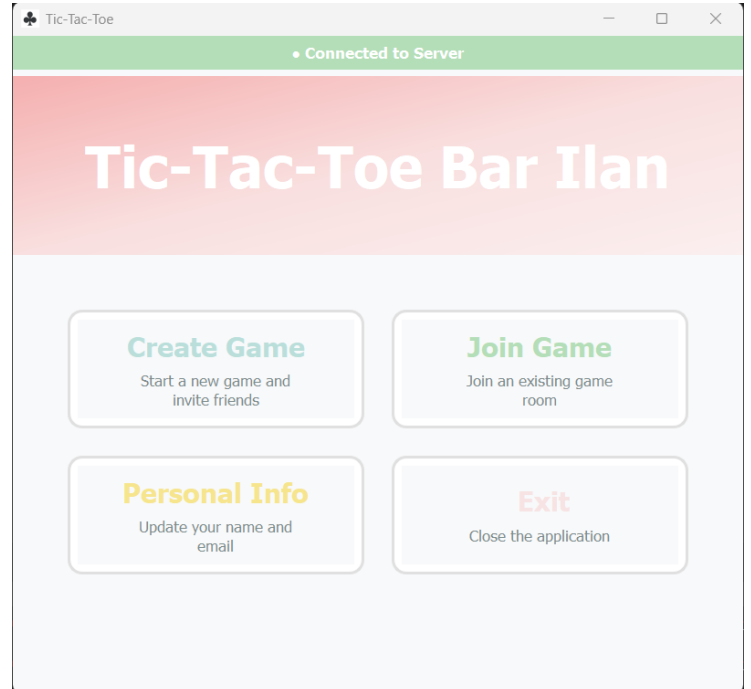
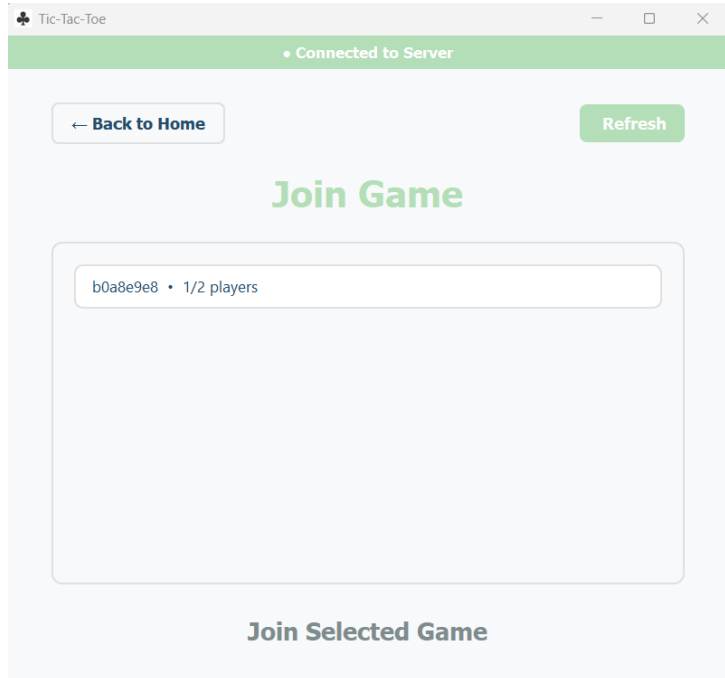
הרצת שרת

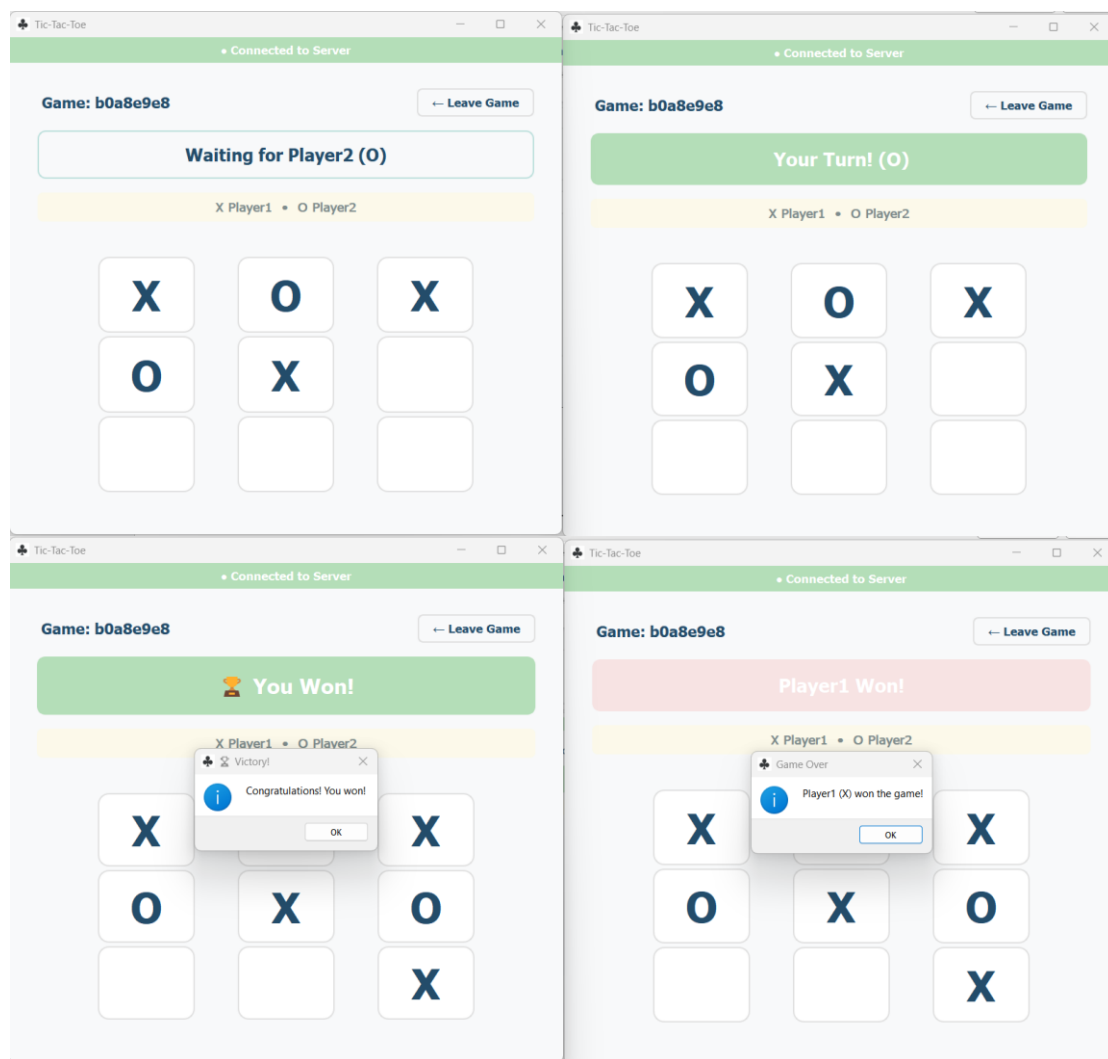
python -m server.main

הרצת לקוח

python -m client.main

11. צילומי מסך





12. סיכום

הפרויקט מדגים תכנון מערכת רשת מלאה עם:

- הפרדה ברורה בין לקוח לשרת
- פרוטוקול תקשורת מסודר
- לוגיקת משחק תקינה
- ממשק משתמש גרפי מקצועי

המערכת יציבה, מודולרית ומתאימה להרחבות עתידיות.